Dual-Layer Web Data Security

Combining AES-256-GCM Encryption & HTML Steganography

A dual-layer approach ensuring both cryptographic confidentiality and data concealment through authenticated encryption and invisible HTML embedding.

Presenter

Aaryan Singh
BTech CSE 'A'

Faculty Advisor

Ms. Vidisha Vishwakarma
Department of Computer Science & Engineering

Made with GAMMA

PROBLEM STATEMENT

The Security Gap: Why Dual-Layer?

What Existing Approaches Miss

Traditional Encryption

Protects data **confidentiality** but reveals the *existence* of secret information to eavesdroppers. An adversary performing traffic analysis immediately knows something secret is being transmitted.

Standard Steganography

Hides data **invisibly** but lacks cryptographic strength — vulnerable to statistical analysis and brute-force attacks. Without encryption, extracted content is immediately readable.

Our Dual-Layer Solution

Layer 1 — Cryptography:
AES-256-GCM authenticated encryption ensures data payload privacy and integrity.

Layer 2 — Steganography:
HTML embedding conceals the encrypted payload inside a normal-looking web document.

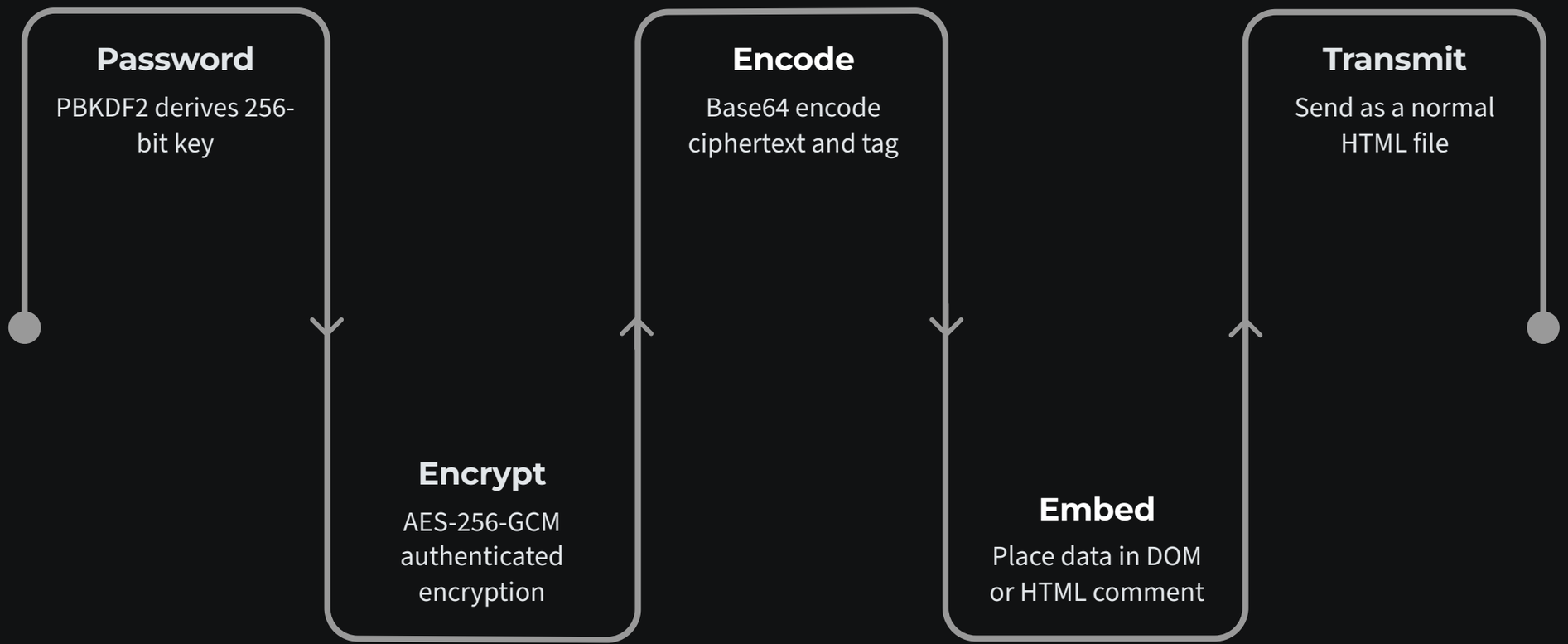
Result: Confidentiality + Concealment = Defense against both *cryptanalysis* and *traffic analysis* simultaneously.

Made with GAMMA

ARCHITECTURE

System Architecture & Data Flow

End-to-end pipeline from plaintext input to secure HTML carrier and back to recovered plaintext.



Sender Side

1. User supplies plaintext data and a password
2. PBKDF2 derives a 256-bit key; AES-256-GCM encrypts the payload
3. Output is Base64-encoded and embedded into an HTML carrier file

Receiver Side

1. HTML file is uploaded and parsed to extract the Base64 payload
2. PBKDF2 re-derives the same key from the shared password
3. AES-256-GCM verifies the auth tag and decrypts to recover plaintext

Made with GAMMA

LAYER 1

Layer 1: Cryptographic Pipeline

1

Key Derivation

PBKDF2-HMAC-SHA256
250,000+ iterations
Derives **256-bit key** from password + random salt.
Resists dictionary and brute-force attacks.

2

Nonce Generation

96-bit CSPRNG Nonce
Cryptographically secure random number generator ensures uniqueness for every individual encryption operation, preventing nonce reuse vulnerabilities.

3

Authenticated Encryption

AES-256-GCM Mode
Produces ciphertext + **128-bit authentication tag**.
Guarantees both confidentiality and integrity. Any tampering is detected and rejected.

Cryptographic Pipeline Diagram

NIST-approved: AES-256-GCM is recommended by NIST SP 800-38D. PBKDF2 with 250,000+ iterations meets OWASP minimum hardening guidelines.

Made with GAMMA

LAYER 2

Layer 2: HTML Steganography & Carrier

- **Step 1 — Binary-to-ASCII Conversion:** The binary ciphertext (including nonce and 128-bit auth tag) is encoded as **Base64**, producing a printable ASCII string safe for embedding directly into HTML markup without encoding errors or structural corruption.
- **Step 2 — Invisible Embedding Methods:**

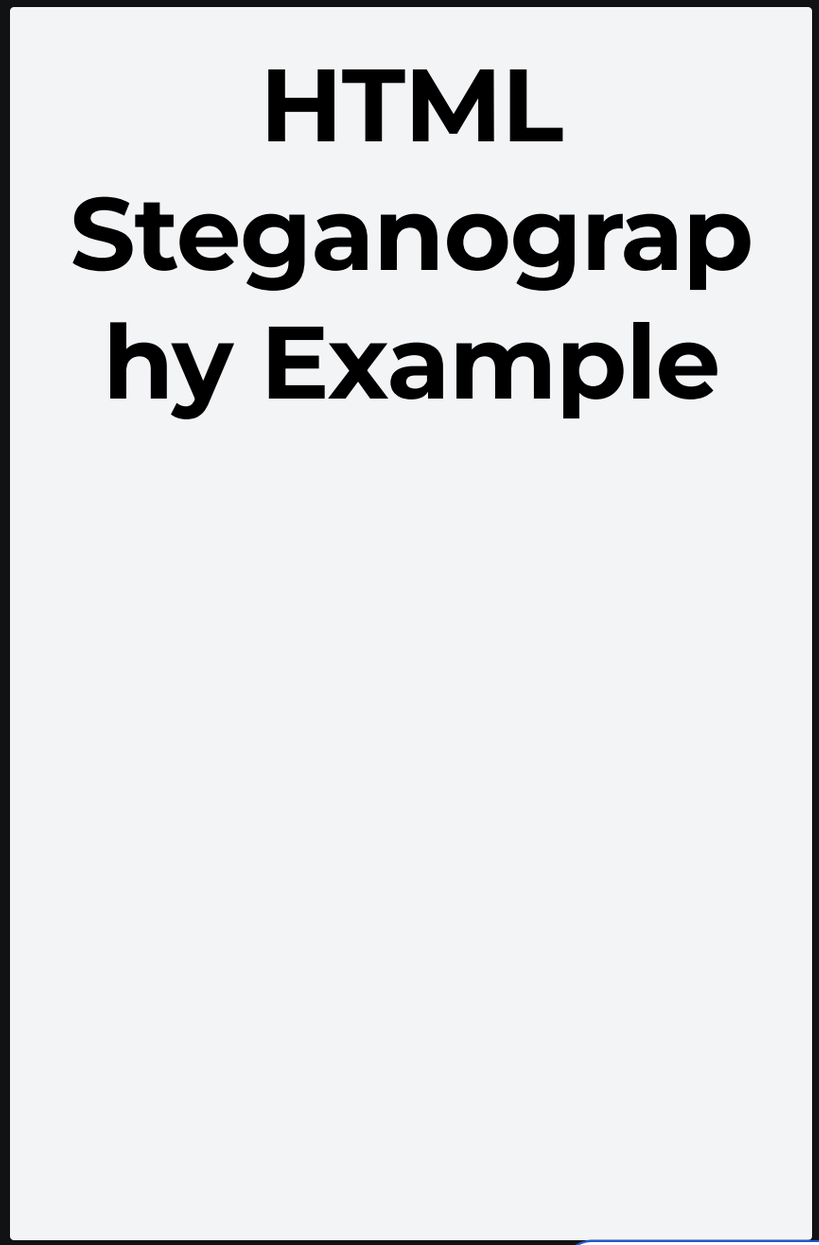
```
<!-- [Base64 payload] -->
```

Option A: HTML Comments

```
<div data-secret="[Base64 payload]">
</div>
```

Option B: Custom DOM Attributes

- Both methods place the payload **outside the visible render tree**. No pixel-level image manipulation required — the carrier is structurally normal HTML5.
- **Step 3 — Carrier File:** The resulting file functions as a **valid, fully renderable HTML document** with no visual or structural anomalies. A browser renders it normally; the hidden payload is entirely invisible to casual inspection — "Security through Obscurity" as a second defensive layer.



Made with GAMMA

DECRYPTION

Receiver-Side Extraction & Decryption

A symmetric four-step process mirrors the encryption pipeline, with the receiver requiring only the shared password and the HTML carrier file.

1

1. Parse HTML

Upload carrier file; extract Base64 payload from HTML comments or `data-*` attributes.

2

2. Base64 Decode

Recover raw binary: **ciphertext**, **96-bit nonce**, and **128-bit authentication tag**.

3

3. Re-derive Key

Receiver enters password; **PBKDF2-HMAC-SHA256** (250,000+ iterations) regenerates the identical 256-bit key using the stored salt.

4

4. Verify & Decrypt

AES-256-GCM verifies the **auth tag**. Rejects if tampered. On success, decrypts to recover original plaintext.

Decryption Flow Diagram

Security Guarantee: Any modification to the ciphertext or authentication tag causes GCM verification to fail — ensuring end-to-end data integrity with no silent corruption.

Made with GAMMA

IMPLEMENTATION

Technical Stack & Implementation

Cryptographic Algorithms

AES-256-GCM

NIST-approved authenticated encryption. Confidentiality + integrity in one pass.

PBKDF2-HMAC-SHA256

Key derivation with 250,000+ iterations. Hardens low-entropy passwords.

SHA-256

Underlying hash function for HMAC in key derivation chain.

Carrier & Embedding Format

- **HTML5** DOM `data-*` attributes / HTML comments
- Base64 ASCII payload — no binary blobs in markup

Implementation Languages

Python — `cryptography` library
Full PBKDF2 + AES-256-GCM + HTML parsing pipeline

JavaScript — `TweetNaCl.js` / `libsodium.js`
Browser-side encryption for progressive web integration

Core Code Modules

1. **encrypt()** — password → PBKDF2 key → AES-256-GCM → Base64 output
2. **embed()** — inject Base64 payload into HTML DOM or comments
3. **extract_decrypt()** — parse HTML → decode Base64 → PBKDF2 re-derive → AES-256-GCM verify & decrypt

System Architecture Diagram

Made with GAMMA

SCOPE & ROADMAP

Applications, Limitations & Future Scope

Use Cases



Untrusted Networks

Secure web communication over public WiFi or compromised channels where traffic analysis is a threat.



Covert Transmission

Data exchange where the very existence of secret information must remain hidden from observers.



Confidential Sharing

Document sharing with plausible deniability — carrier appears as an ordinary HTML page.

Current Limitations

Payload Size

Restricted by HTML file size and chosen embedding method. Large payloads increase carrier file size noticeably.

Processing Latency

PBKDF2 at 250,000+ iterations introduces measurable delay on low-power or embedded devices.

Steganalysis Risk

Steganography alone does not prevent detection by advanced statistical steganalysis tools.

Future Roadmap

- Extend carrier support to **CSS and JavaScript** files
- Integration with **dynamic web apps and PWAs**
- Adaptive payload sizing with **compression**
- Explore **post-quantum cryptographic algorithms** (CRYSTALS-Kyber, CRYSTALS-Dilithium)

Made with GAMMA